Task 2: Attention Mechanisms and Transformer Encoders

1 Introduction

In this assignment, the goal was to understand how attention works inside a Transformer encoder. Instead of focusing only on getting high accuracy, the main aim was to see how the model learns and how attention behaves. A single-layer Transformer encoder was built from scratch and tested on a text classification task.

2 Dataset and Tokenizer

We used a subset of the AG News dataset with 500 samples from four classes: World, Sports, Business, and Sci/Tech. Each sentence was limited to 32 tokens.

2.1 Vocabulary

A tokenizer was built from scratch. Compared to character-level tokenization, word-level tokenization resulted in a larger vocabulary but was more meaningful. Words directly carry semantic information, so the model was able to learn better patterns using them.

2.2 Patterns in Different Classes

While observing the dataset, some clear patterns were seen:

- Sports articles had words like "team", "win", and "game"
- Business articles included "market", "stocks", and "company"
- Sci/Tech articles used words like "technology" and "software"
- World news contained words like "government" and "war"

These patterns helped the model during training.

2.3 Effect of Sequence Length

Since all inputs were limited to 32 tokens, some longer sentences were cut. In a few cases, important words at the end were removed, which could affect classification performance.

3 Model Implementation

The Transformer encoder was implemented using basic PyTorch operations without using any built-in transformer modules.

3.1 Attention Mechanisms

Scaled dot-product attention was implemented first, using queries, keys, and values. Then self-attention was applied, where the same sequence is used for all three. Finally, multi-head attention was added, allowing the model to learn different types of relationships at the same time.

3.2 Encoder Structure

The encoder block included:

- Multi-head attention
- Add and layer normalization
- Feed-forward network
- Add and layer normalization

Mean pooling was used at the end to convert token outputs into a single vector for classification.

4 Training

The model was trained using cross-entropy loss. The loss decreased over epochs, but not very smoothly. There were small fluctuations, which shows that the model was learning but not perfectly stable. Compared to ideal behavior, the loss remained slightly higher, indicating that the model did not fully converge.

5 Task 3: Attention Heatmaps

Attention heatmaps were generated for five different sentences.

5.1 Observations

The attention patterns were mixed. In some cases, the model focused on important words like “market” or “team”. However, in many cases, the attention was spread out and did not show a very clear pattern.

5.2 Bad Attention Patterns

Bad attention patterns looked almost uniform, where all tokens had similar attention values. Some heads also showed random scattered patterns. These indicate that the model has not learned strong relationships between words.

6 Task 4: Head Comparison

With 4 attention heads, each head behaved slightly differently, but not perfectly.

- One head sometimes focused on important keywords
- One head captured nearby word relationships
- One head showed broader but unclear attention
- One head appeared mostly random

6.1 Analysis

Multiple heads are useful because they can learn different patterns. However, in this case, not all heads specialized clearly. Some heads seemed redundant or noisy, which shows that increasing heads does not always guarantee better learning.

7 Task 5: Causal Masking

When causal masking was applied, tokens could only attend to previous tokens and not future ones.

7.1 Observations

This restricted the flow of information. Tokens later in the sentence had less context available.

7.2 Why It Matters

This is important for text generation tasks, where future words should not be known in advance.

7.3 Effect on Classification

For classification, this restriction is not helpful. Since the model needs full context, using causal masking would likely reduce performance.

8 Task 6: Cross-Attention

Cross-attention was tested by using one sequence as input and another as target.

8.1 Observations

Some attention connections made sense, where words in the second sequence focused on related words in the first. However, many connections were weak or unclear, especially when the meaning was not straightforward.

9 Task 7: Positional Encoding

Sinusoidal positional encoding was added so the model could understand word order.

9.1 Observations

The positional encoding showed smooth repeating patterns. Each position had a unique representation. Rows correspond to positions in the sequence, and columns represent different dimensions.

10 Ablation Studies

10.1 Removing Scaling

Without scaling, attention values became too large. This caused unstable behavior in the softmax function and made training less stable.

10.2 Removing Positional Encoding

Without positional encoding, the model struggled to understand the order of words. When the same sentence was shuffled, the model produced similar attention patterns, showing that it was not sensitive to order.

10.3 Changing Number of Heads

Using different numbers of heads showed that:

- 1 head was too limited
- 2 heads improved learning slightly
- 4 heads gave better results, but some heads were still not useful

11 Written Analysis

11.1 1. Encoder vs MLP

A Transformer encoder is better than an MLP for text because it can capture relationships between words. An MLP treats inputs independently, while attention allows the model to focus on important words depending on the context.

11.2 2. Head Interpretation

One attention head showed some focus on important words, but the pattern was not always clear. Some heads looked meaningful in certain cases, while others appeared random. This suggests that the model partially learned useful patterns but not completely.

11.3 3. Order Sensitivity

When positional encoding was removed, the model became much less sensitive to word order. It was not completely blind, but it struggled to distinguish between correct and shuffled sentences.

11.4 4. Encoder vs Autoencoder

The encoder produces a vector for each token, which keeps more detailed information. In contrast, an autoencoder compresses the entire input into one vector, which can lose important details. So the encoder representation is richer.

11.5 5. Failure Analysis

In a misclassified example, the attention was spread across many tokens instead of focusing on key words. Because of this, the model could not clearly identify the important features of the sentence, leading to an incorrect prediction.

12 Conclusion

This assignment showed how attention works in practice. Multi-head attention allows the model to learn different patterns, but not all heads are equally useful. Positional encoding is important for understanding order, and scaling helps keep training stable. Overall, the model was able to learn some useful patterns.